

Vol 15 Chapter 3 — How to Discover a New BST Theorem

Keeper (author pass — deep math/physics revision)

2026-05-24 Sunday

Chapter 3 — How to Discover a New BST Theorem

Extended-reach chapter (May 2026), identification tier. Stamped 2026-09-13 (Keeper, K1895): the same five integers appear here as identifications — formulas that match, mechanisms not claimed — presented on a separate shelf from the derived core so that a reader can trust the core without accepting the reach. Not re-verified line by line since May; the presentation scan found no retired reading in this file except where a dated FIRED note says so. Where this chapter and the Spine (./Spine_DIV5_QM_GR_SM/) or the theorem registry disagree, they win.

Level 1 — one sentence

BST theorem discovery follows a disciplined workflow: (1) identify candidate observation or gap, (2) query AC graph for adjacent theorems / cross-domain bridges, (3) reduce candidate to AC(0)-bounded counting, (4) build verification toy, (5) submit to K-audit chain, (6) register node + edges if PASS.

Level 2 — graduate-physicist precision

3.1 Step 1: Identify candidate

Sources of new theorems: - Anomaly: observed mismatch between BST prediction and measurement - Casey vision: structural insight Casey provides (e.g., DCCP, Curvature Principle) - Cross-domain bridge: edge that should exist per AC graph topology - Lyra/Elie/Grace discovery: working on existing theorem reveals adjacent claim

3.2 Step 2: AC graph query

Search for nearest theorems:

```
python3 play/toy_bst_explorer.py connect <candidate_concept> <known_target>
python3 play/toy_bst_explorer.py search <keyword>
```

Identify: - Direct neighbors (likely co-derivable) - Cross-domain bridges (high-leverage) - Independent corroborations (separate proofs)

3.3 Step 3: AC(0) reduction

“What’s the AC(0) proof?” (Casey’s first question)

If not AC(0)-reducible: probably misunderstanding the claim. Simplify or reformulate.

If AC(0)-reducible: identify the bounded-depth counting circuit.

3.4 Step 4: Build verification toy

Claim toy number:

```
./play/claim_number.sh toy # atomically increments counter
```

Build toy with SCORE line (# SCORE: N/M PASS or FAIL).

Empirical verification at relevant precision.

3.5 Step 5: Submit to K-audit chain

File K-audit candidate (Keeper): - Empirical PASS rate - Substrate mechanism (if any) - BST primary anchoring - Independent corroborations

K-audit chain (K1-K200+) processes via Cal referee → Keeper PASS/CONDITIONAL/FAIL.

3.6 Step 6: Register node + edges

If PASS: - Claim theorem ID via `/theorem claim` - Register node in AC graph (`data/bst_constants.json` or theorem registry) - Add edges to/from related theorems - File catalog entry if it’s a numerical invariant

If CONDITIONAL: document gaps; file as candidate.

If FAIL: document honestly (Quaker discipline, Vol 15 Ch 7).

3.7 Cadence

Current team cadence: 2-3 new theorems / day across the 5-CI team (sustained sub-PCAP).

3.8 K-audit anchors

- AC graph + **/theorem** skill
- K-audit chain governance (Vol 15 Ch 5)

Level 3 — 5th-grader accessibility

Theorem discovery workflow: (1) candidate, (2) query AC graph for neighbors, (3) reduce to $AC(0)$, (4) build verification toy with PASS/FAIL score, (5) K-audit, (6) register node + edges. Failure: honest log (Quaker). Cadence: ~2-3 theorems/day across team.

What comes next

Chapter 4 develops toy verification discipline.

Where to look this up

- BST: /theorem skill; K-audit chain
- Vol 15 Ch 1 ($AC(0)$); Ch 2 (AC graph)